

Zadanie 03 - Natalia Włodyka

Polecenie zadania:

Celem zadania jest znalezienie dwóch największych na moduł wartości własnych, oraz ich wektorów własnych, danej w zadaniu macierzy. Stosujemy celu znalezienia pierwszej metodę potęgową, a do znalezienia drugiej wykorzystujemy wektor pierwszej własności własnej oraz iterację.

Zadana macierz:

W zadaniu wyliczamy dwie największe wartości własne podanej symetrycznej macierzy A:

$$A = \begin{pmatrix} \frac{19}{12} & \frac{13}{12} & \frac{5}{6} & \frac{5}{6} & \frac{13}{12} & -\frac{17}{12} \\ \frac{13}{12} & \frac{13}{12} & \frac{5}{6} & \frac{5}{6} & -\frac{11}{12} & \frac{13}{12} \\ \frac{5}{6} & \frac{5}{6} & \frac{5}{6} & -\frac{1}{6} & \frac{5}{6} & \frac{5}{6} \\ \frac{5}{6} & \frac{5}{6} & -\frac{1}{6} & \frac{5}{6} & \frac{5}{6} & \frac{5}{6} \\ \frac{13}{12} & -\frac{11}{12} & \frac{5}{6} & \frac{5}{6} & \frac{13}{12} & \frac{13}{12} \\ -\frac{17}{12} & \frac{13}{12} & \frac{5}{6} & \frac{5}{6} & \frac{13}{12} & \frac{19}{12} \end{pmatrix}$$

Metoda potęgowa:

Do znalezienia największej wartości własnej wykorzystujemy metodę potęgową, którą stosujemy wobec macierzy symetrycznych i diagonalizowalnych. W kodzie implementujemy podaną iterację:

Dla $y_1 = 1$, poza tym dowolny:

$$\begin{aligned} Ay_k &= z_k \\ y_{k+1} &= \frac{z_k}{\|z_k\|} \end{aligned}$$

Znalezienie drugiej wartości własnej:

Do znalezienia drugiej wartości wykorzystujemy pierwszy wektor własny oraz podaną iterację:

Iteracja ($\|y_1\| = 1$, $e_1^T y_1 = 0$, poza tym dowolny)

$$\begin{aligned} Ay_k &= z_k \\ z_k &= z_k - e_1(e_1^T z_k) \\ y_{k+1} &= \frac{z_k}{\|z_k\|} \end{aligned}$$

Kod w Pythonie:

```
from typing import Any

import numpy as np
from numpy.typing import NDArray

vector = NDArray[np.float64]
matrix = NDArray[np.float64]

def power_method(A: matrix, iterations: int) -> tuple[vector, np.float64]:
    y = np.random.rand(A.shape[0]) #Tworzymy randomowy wektor
    y = y/ np.linalg.norm(y) #Normalizujemy go
```

```

    for _ in range(iterations):
        #Obliczamy wektor Ay
        z = A @ y

        #Obliczamy jego norme
        norm_of_z = np.linalg.norm(z)

        #Normalizuje wektor
        y = z / norm_of_z

    #Wyliczam wartosc wlasna
    value = float(norm_of_z)

    return y, value

def second_value(A: matrix, first: vector, iterations: int) -> tuple[vector,
np.float64]:
    y = np.random.rand(A.shape[0])
    y -= first * (first @ y) # rzutowanie
    y = y / np.linalg.norm(y)

    for _ in range(iterations):
        z = A @ y
        z = z - first*(first @ z)
        norm_of_z = np.linalg.norm(z)
        y = z / norm_of_z

    value = float(norm_of_z)

    return y, value

def main():
    A: vector = np.array([[19 / 12, 13 / 12, 5 / 6, 5 / 6, 13 / 12, -17 / 12],
                          [13 / 12, 13 / 12, 5 / 6, 5 / 6, -11 / 12, 13 / 12],
                          [5 / 6, 5 / 6, 5 / 6, -1 / 6, 5 / 6, 5 / 6],
                          [5 / 6, 5 / 6, -1 / 6, 5 / 6, 5 / 6, 5 / 6],
                          [13 / 12, -11 / 12, 5 / 6, 5 / 6, 13 / 12, 13 / 12],
                          [-17 / 12, 13 / 12, 5 / 6, 5 / 6, 13 / 12, 19 / 12]],
dtype=np.float64)
    first_vector, first_value = power_method(A, 1000)
    second_vector, value_second = second_value(A, first_vector, 1000)
    print("Wyswietlamy wyniki")
    print(f"Pierwszy wektor: {first_vector}")
    print(f"Pierwsza wartosc wlasna: {first_value}")
    print(f"Drugi wektor: {second_vector}")
    print(f"Druga wartosc wlasna: {value_second}")

main()

```

Wyniki:

Pierwszy wektor: [0.40824829 0.40824829 0.40824829 0.40824829 0.40824829 0.40824829]

Pierwsza wartość własna: 4.0

Drugi wektor: [-7.07106781e-01 1.04231312e-16 6.72238776e-17 6.72238776e-17
1.04231312e-16 7.07106781e-01]

Druga wartość własna: 3.0000000000000004

Omówienie wyników:

W zaokrągleniu wartości własne wynoszą $\lambda_1 = 4$ oraz $\lambda_2 = 3$. Jak widać z powyższego wyniku oraz kodu, metoda potęgowa dla wystarczająco dużej liczbie iteracji będzie dążyć do wektora własnego największej wartości własnej.